

Python 版 AIPL ユーザーズガイド

児玉靖司 (Yasushi Kodama)

法政大学 (Hosei University)

2026 年 5 月 12 日

目次

1	概要	2
2	対象実装	2
3	全体構成	2
4	基本言語機能	2
5	型システム	3
6	Phase 11 から 17 の研究機能	3
7	データ構造	4
8	関数	4
9	アクター機能	4
10	チャンネル	4
11	動的コンパイルと動的生成	4
12	AI 連携	4
13	ファイル、JSON、画像処理	5
14	分散アクター	5
15	モデル検査	6
16	実行方法	6
17	サンプルとテスト	6
18	まとめ	6

1 概要

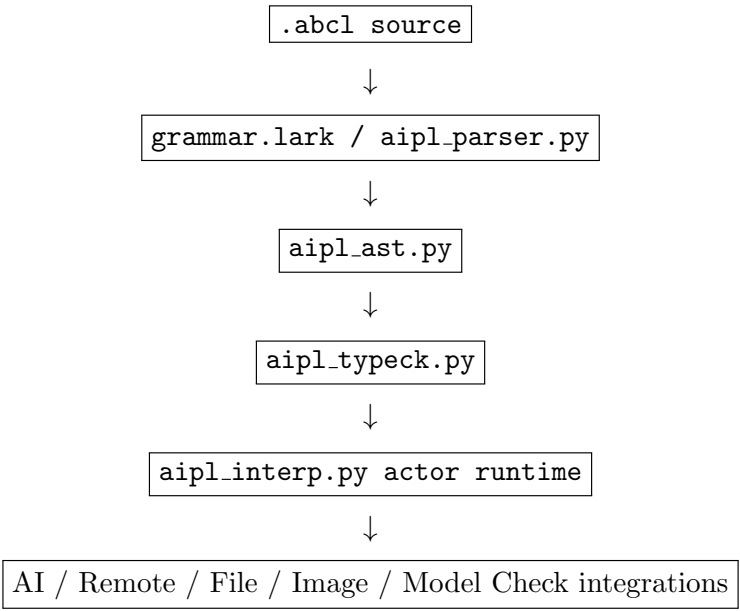
Python 版 AIPL は、src/python-aipl/ にある研究・拡張用ランタイムである。OCaml 版よりも新しい言語機能を多く持ち、型注釈、効果注釈、所有権、線形型、構造化並行、動的コンパイル、AI 呼び出し、モデル検査などを扱える。

Python 版 AIPL は単なる ABCL 風アクター処理系ではなく、型付きアクター言語、AI 統合言語、分散アクター実験基盤、モデル検査対象言語、動的自己拡張言語として利用できる。

2 対象実装

項目	ファイル
ランタイム本体	src/python-aipl/aipl_interp.py
パーサ/構文	src/python-aipl/grammar.lark, src/python-aipl/aipl_parser.py
AST	src/python-aipl/aipl_ast.py
型検査	src/python-aipl/aipl_typeck.py
実行入口	src/python-aipl/aipl_main.py
サンプル	src/python-aipl/samples/, samples-ai/, samples-remote/, samples-mc/

3 全体構成



4 基本言語機能

Python 版 AIPL は、class、method、var、function を中心にしたアクター指向言語である。クラスから生成されたインスタンスはアクターとして動作し、send、now、future、await によって通信する。

構文	意味
----	----

<code>send actor.method(args);</code>	非同期送信。呼び出し元は待たない。
<code>now actor.method(args)</code>	同期呼び出し。相手の <code>reply</code> を待つ。
<code>future actor.method(args)</code>	Future を返す非同期呼び出し。
<code>await(f) / await f</code>	Future の結果を待つ。
<code>self</code>	現在のアクター自身。
<code>sender</code>	メッセージ送信元。
<code>become Class(args);</code>	現在のアクターの振る舞いを別クラスへ変更する。

5 型システム

Python 版 AIPL は、OCaml 版より新しい型付き構文を持つ。変数、引数、戻り値に型注釈を書ける。

```
method add(a: int, b: int) -> int {
  return a + b;
}
```

対応する主な型を以下に示す。

- `int`
- `float`
- `string`
- `bool`
- `array[T]`
- `tuple(...)`
- `record{...}`
- Union 型
- 型変数
- 長さ付き配列
- `linear T`

`typeof(x)` により実行時の構造型を確認できる。

6 Phase 11 から 17 の研究機能

Phase	機能
Phase 11	型注釈、Union 型、Generics、長さ付き配列。
Phase 12	capability effects。例: <code>!{fs, ai, net, mut}</code> 。
Phase 13	CSP 風チャンネル。
Phase 14	<code>linear</code> による線形値、use-after-move 検出。
Phase 15	<code>pub</code> フィールドと所有権、外部書き込み禁止。
Phase 16	transient cast、 <code>any -> T</code> 境界の実行時検査。
Phase 17	<code>scope { ... }</code> による構造化並行。

7 データ構造

配列、タプル、レコードを扱える。配列は `[1, 2, 3]` のようなリテラル、`array_push`、`array_get`、`array_set` など操作する。多次元配列もあり、`var grid[rows][cols];` のように動的サイズで宣言できる。

タプルは `(1, "a")`、レコードは `{ name: "Alice", age: 30 }` のように書ける。レコードは `r.name` のようなドットアクセスに対応している。

8 関数

トップレベル関数とクラス内関数を定義できる。関数は `return` によって同期的に値を返す。クラス内関数は、そのクラスのメソッドから補助関数として呼び出せる。

`typeof(function_name)` により、実行時に観測された呼び出し履歴から推定された関数シグネチャを表示できる。

9 アクター機能

アクターはメッセージキューを持ち、非同期に動作する。`reply(value)` によって `now` や `future` の呼び出し元に値を返す。

`become Class(args);` によって、実行中のアクターの振る舞いを別クラスへ変更できる。

10 チャンネル

CSP 風のチャンネル機能がある。アクター間通信とは別に、明示的な同期キューを使える。

```
channel(capacity, type_name)
channel_send(ch, value)
channel_recv(ch)
channel_try_recv(ch)
channel_close(ch)
channel_size(ch)
select_recv([ch1, ch2], timeout_ms)
```

11 動的コンパイルと動的生成

`compile(source)` により、文字列として渡した AIPL ソースを実行中にコンパイルできる。`spawn(class_name, args...)` により、文字列で指定したクラスからアクターを生成できる。

さらに、以下のメソッドインジェクション機能がある。

```
add_method(target, source)
remove_method(target, name)
methods_of(target)
```

これにより、クラスまたは特定アクターへメソッドを動的に追加・削除できる。

12 AI 連携

Python 版 AIPL は、生成 AI 呼び出しを組み込み関数として持っている。

```
ai_call(prompt)
```

```
ai_call_with_system(system, prompt)
ai_call_priority(priority, prompt)
ai_call_retry(max_attempts, prompt)
ai_call_image(prompt, image)
ai_usage()
ai_remaining()
ai_cost()
```

Gemini、Anthropic Claude、OpenAI を切り替えられる。AIPL_AI_PROVIDER=mock を使うと、外部 API なしでテストできる。

また、起動時に AI アクターが自動生成され、now AI.ask("...") や future AI.ask("...") の形で AI をアクターとして扱える。

13 ファイル、JSON、画像処理

Python 版 AIPL には、アプリ生成やデータ処理向けの組み込みもある。

関数	概要
read_file	テキストファイルを読む。
write_file	テキストファイルを書く。
append_file	テキストファイルへ追記する。
read_bytes	バイナリを読む。
write_bytes	バイナリを書く。
json_parse	JSON 文字列を AIPL 値へ変換する。
json_stringify	AIPL 値を JSON 文字列へ変換する。
image_load	画像を読み込む。
image_save	画像を保存する。
image_create	画像を生成する。
image_pixel	ピクセル値を読む。
image_set_pixel	ピクセル値を書き込む。
image_size	画像サイズを返す。

画像処理は Pillow を使う。AIPL だけで HTML、CSS、画像、JSON manifest を生成するサンプルもある。

14 分散アクター

HTTP 経由で別プロセス、別ランタイムのアクターと通信できる。

```
web_listen(port)
web_expose(name, actor)
remote_call(host, actor, method, args...)
remote_now(host, actor, method, args...)
remote_future(host, actor, method, args...)
serve_forever()
```

OCaml 版との相互通信も想定されており、JSON 形式の /api/json/send と /api/json/call を使う。

15 モデル検査

Python 版 AIPL には、アクタープログラムの bounded model checking 用モジュールがある。`aipl_modelcheck.py`、`aipl_modelcheck_load.py` により、食事する哲学者やロック順序などのサンプルを検査できる。

また、Promela/SPIN 向け出力と TLA+ 向け出力もある。

- `aipl_to_promela.py`
- `aipl_to_tla.py`
- `samples-mc/*.abcl`

16 実行方法

Python 版 AIPL のメイン実行ファイルは次である。

```
src/python-aipl/aipl_main.py
```

この環境ではラッパーとして次も使っている。

```
scripts/run_aipl_python313.sh
```

実行例:

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-s-claude
scripts/run_aipl_python313.sh src/python-aipl/samples/Hello.abcl
```

17 サンプルとテスト

サンプルは主に以下にある。

- `src/python-aipl/samples/`
- `src/python-aipl/samples-ai/`
- `src/python-aipl/samples-remote/`
- `src/python-aipl/samples-mc/`

テストは `_test_*.py` として多数用意されており、型検査、配列、タプル、レコード、チャンネル、線形型、所有権、メソッドインジェクション、AI アクター、モデル検査などを確認できる。

18 まとめ

Python 版 AIPL は、型付きアクター言語としての中核に加え、AI 統合、動的拡張、分散アクター、モデル検査、ファイル・画像・JSON 処理を統合した実験用言語である。OCaml 版がネイティブ実行とアクターランタイムの基礎を担う一方で、Python 版は新しい言語仕様と AI 連携を素早く検証するための中心的な実装である。